CS100 Introduction to Programming
Fall 2025
Midterm Exam

Instructors: Ying Cao

Time: December 16th 13:00 - 14:50

INSTRUCTIONS

Please read and follow the following instructions:

- You have 100 minutes to answer the questions.
- You are not allowed to bring any electronic devices including regular calculators.
- You are not allowed to discuss or share anything with others during the exam.
- You should write the answer to every problem in the dedicated box **clearly**.
- You should write **your name and your student ID** as indicated on the top of **each page** of the exam sheet.

Name	
Student ID	

1. (20 points) Undefined behavior and sanitizers

Consider the following C program (saved in a file `main.c`):

```c
#include <stdlib.h>

int **create2Darray(int m, int n)
{
    int **p = malloc(sizeof(int *) * n);
    for (int i = 0; i != n; ++i)
        p[i] = malloc(sizeof(int) * m);
    return p;
}
void free2Darray(int **pArray, int n)
{
    free(pArray);
    for (int i = 0; i != n; ++i)
        free(pArray[i]);
}
int main(void)
{
    int **array_2D = create2Darray(3, 4);
    free2Darray(array_2D, 4);
}
```

(1) (4') On an Ubuntu 24.04 Linux system, how do you compile this program with `gcc` (without any extra warning or sanitizer options) and produce an executable file named `main`? Write a single command line.
`gcc main.c -o main`

(2) (4') The code above compiles successfully, but when you run it on an Ubuntu 24.04 Linux system, it prints `Segmentation fault (core dumped)` This suggests that the program may have undefined behavior. We want to use a sanitizer to help detect the cause (assume we use GCC and AddressSanitizer). How do you **compile** and **run** the program with AddressSanitizer enabled?
Compile: `gcc -fsanitize=address -g main.c -o main`
Run: `./main`

(3) (2') Is this kind of sanitizer-based undefined behavior detection done at *compile time* or *run time*? (Answer with exactly `compile time` or `run time`.) `run time`

(4) Suppose we compile and run the program with AddressSanitizer, and see an error message:

```
==17610==ERROR: AddressSanitizer: heap-use-after-free on address 0x503000000040
READ of size 8 at 0x503000000040 thread T0
    #0 0x62a2997d02d8 in free2Darray main.c:14
    #1 0x62a2997d0323 in main main.c:19
    #2 0x7e4b53e2a1c9  (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9)
    #3 0x7e4b53e2a28a in __libc_start_main
    #4 0x62a2997d0124 in _start
READ of size 8 at 0x503000000040 thread T0
freed by thread T0 here
    #0 0x729eabefc4d8 in free ../src/libsanitizer/asan/asan_malloc_linux.cpp:52
    #1 0x5838cf28929b in free2Darray main.c:12
    #2 0x5838cf289323 in main main.c:19
previously allocated by thread T0 here:
    #0 0x729eabefd9c7 in malloc ../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x5838cf28920c in create2Darray main.c:5
    #2 0x5838cf28930e in main main.c:18
SUMMARY: AddressSanitizer: heap-use-after-free main.c:14 in free2Darray
```

i. (4') What is the cause of the error? (Hint: the sanitizer reports `heap-use-after-free`.)

Solution: The function `free2Darray` frees `pArray` first, so later accesses to `pArray[i]` use memory that has already been freed.

ii. (6') Fix the error.

Solution: Free the inner arrays first, then free `pArray`:
`for (int i = 0; i != n; ++i) free(pArray[i]); free(pArray);`

2. (16 points) Code debugging

Consider the following five code snippets. For each, determine if the code is correct or incorrect. If correct, write `true`. If incorrect, write `false` and briefly explain the error.(Assume we have include the necessary headers and definitions.)

(1) (4') Size of struct (C):

```c
struct A {
    int x;
    struct A a;
};
```

Correct? false

If incorrect, explain:

Solution: The code is incorrect because it defines a struct 'A' that contains a member of its own type, which leads to an infinite size. In C, this is not allowed as it creates an incomplete type.

(2) (4') Copy assignment operator (C++):

```cpp
class Dynarray {
    public:
        Dynarray &operator=(const Dynarray &other) {
            auto new_data = new int[other.m_length];
            std::copy_n(other.m_storage, other.m_length, new_data);
            m_storage = new_data;
            m_length = other.m_length;
            return *this;
        }
};
```

Correct? false

If incorrect, explain:

Solution: The code is incorrect because the assignment operator does not free the existing memory allocated to 'm_storage', leading to a memory leak.

(3) (4') Copy-and-swap (C++):

```cpp
class Dynarray {
    public:
        void swap(Dynarray &other) noexcept {
            std::swap(m_storage, other.m_storage);
            std::swap(m_length, other.m_length);
        }
```

```
7           Dynarray &operator=(const Dynarray &other) {
8               Dynarray(other).swap(*this); // C++23: auto{other}.swap(*this);
9               return *this;
10          }
11  }
```

Correct? true

If incorrect, explain:

Solution:

(4) (4') Standard library algorithms (C++):

```
1  std::vector<int> a = someValues();
2  std::vector<int> b(a.size());
3  std::vector<int> c{};
4  std::copy(a.begin(), a.end(), b.begin());
5  std::copy(a.begin(), a.end(), c.begin());
```

Correct? false

If incorrect, explain:

Solution: The code is incorrect because the vector 'c' is initialized as an empty vector, so it does not have enough space to hold the copied elements from vector 'a'. This will lead to undefined behavior. To fix this, 'c' should be resized to match the size of 'a' before copying.

3. (20 points) Lvalues, rvalues, copy, and move

Consider the following C++ code:

```
1  #include <string>
2  #include <utility>
3
4  class Message {
5      std::string m_contents;
6  public:
7      explicit Message(const std::string &contents) : m_contents{contents} {}
8  };
9
10 std::string s1 = foo(), s2 = bar();
11 Message m1(s1);
12 Message m2(s1 + s2);
```

(1) (4') What is the use of `explicit` in line 7? What differences will occur if we remove `explicit`? (you can provide an example to illustrate.)

Solution: If we remove 'explicit', the constructors can be used for implicit conversions. For example:

```
1  Message m = "hello";
2  Message n = std::string("world");
```

(2) (8') What happens if we remove `const` from line 7? Answer from the perspective of passing lvalues and rvalues.

Solution: If we remove 'const' from line 7, we can no longer bind an rvalue (temporary) to the parameter it will cause compile error, as non-const lvalue references cannot bind to rvalues. This means we would not be able to construct a 'Message' from a temporary string created by 's1 + s2'. For example:

```
Message m(s1 + s2);
```

For const lvalues, it will cause compile error as well, since non-const lvalue references cannot bind to const lvalues. For non-const lvalues, it may be dangerous to allow modification of the original string passed in, which is generally not desired when constructing a new object.

(3) (8') How can we modify the code above to optimize for copy on lvalues and move on rvalues with two different ways? (hint: one way is adding `move constructor`, another use `Pass-by-value`)

Solution:

```
class Message {
    std::string m_contents;
    public:
        explicit Message(std::string contents) : m_contents{std::move(contents)} {}
};
class Message {
    std::string m_contents;
    public:
        explicit Message(const std::string &contents) : m_contents{contents} {}
        explicit Message(std::string &&contents) : m_contents{std::move(contents)} {}
};
```

4. (15 points) Inheritance and Polymorphism

Consider the following classes `Bird`, `Sparrow`, and `Penguin`:

```
class Bird {
public:
    Bird() = default;
    virtual void fly() const = 0; // Pure virtual function, implying that all birds can fly
    virtual ~Bird() = default;
};

class Sparrow : public Bird {
public:
    Sparrow() = default;
    ~Sparrow() = default;
    void fly() const override {
        std::cout << "Sparrow flies" << std::endl;
    }
};

class Penguin : public Bird {
public:
    Penguin() = default;
    ~Penguin() = default;
    void fly() const override {
        std::cout << "Penguins can't fly" << std::endl;
```

```
    }
};
```

Upon re-evaluation, it becomes clear that not **all** birds can fly, which leads to a design inconsistency. Follow the tasks below to refactor the code accordingly:

(1) (5') What is printed when the following code is executed?

```
Bird* bird1 = new Sparrow();
Bird* bird2 = new Penguin();
bird1->fly();
bird2->fly();
delete bird1;
delete bird2;
```

Solution:

```
Sparrow flies
Penguins can't fly
```

(2) (10') The current design where `Bird` has a `fly` method is not optimal. Refactor the class hierarchy to improve it. Here are some guidelines:

- Create a new abstract class `FlyingBird`, which inherits from `Bird`, for birds that can fly.
- The class `Sparrow` should inherit from `FlyingBird`.
- The `Penguin` class should remain a direct subclass of `Bird` since it cannot fly.
- Implement default constructors and destructors as needed.

Solution:

```
class Bird {
public:
    Bird() = default;
    virtual ~Bird() = default;
};

class FlyingBird : public Bird {
public:
    FlyingBird() = default;
    virtual void fly() const = 0; // Only flying birds have this behavior
    virtual ~FlyingBird() = default;
};

class Sparrow : public FlyingBird {
public:
    Sparrow() = default;
    ~Sparrow() = default;
    void fly() const override {
        std::cout << "Sparrow flies" << std::endl;
    }
};

class Penguin : public Bird {
public:
```

```
    Penguin() = default;
    ~Penguin() = default;
};
```

5. (19 points) Rational Class

Consider the following C++ code:

```
#include <iostream>
#include <numeric> // For std::gcd
class Rational {
    int m_num;     // Numerator. The sign is only on the numerator.
    unsigned m_denom;  // Denominator

    void simplify() {  // Private, because this is our implementation detail.
        int gcd = std::gcd(m_num, m_denom); // std::gcd in <numeric> (since C++17)
        m_num /= gcd;
        m_denom /= gcd;
    }

public:
    Rational(int x = 0) : m_num{x}, m_denom{1} {} // Also a default constructor.
    Rational(int num, unsigned denom) : m_num{num}, m_denom{denom} { simplify(); }

    Rational(double d) {
        const unsigned precision = 1000000;
        m_num = static_cast<int>(d * precision);
        m_denom = precision;
        simplify();
    }

    operator double() const {
        return static_cast<double>(m_num) / m_denom;
    }

    Rational &operator+=(const Rational &rhs) {
        m_num = m_num * static_cast<int>(rhs.m_denom) + static_cast<int>(m_denom)
        * rhs.m_num;
        m_denom = m_denom * rhs.m_denom;
        simplify();
        return *this;
    }

    friend Rational operator+(const Rational &, const Rational &);
    friend std::ostream &operator<<(std::ostream &, const Rational &);
};

Rational operator+(const Rational &lhs, const Rational &rhs) {
    return Rational(
        lhs.m_num * static_cast<int>(rhs.m_denom)
            + static_cast<int>(lhs.m_denom) * rhs.m_num,
        lhs.m_denom * rhs.m_denom
    );
```

```
46 }
```

(1) (2') The function `operator+` declared with the keyword `friend` inside `Rational` is **member functions** of `Rational`. (True/False) `False`

> **Solution:** It is **friend non-member functions**. Being declared inside the class with `friend` does not make them member functions.

(2) (2') It is possible in C++ to overload the `**` operator to implement exponentiation for `Rational`. (True/False) `False`

(3) (3') Consider the following code:

```
1 Rational a{1, 2};
2 Rational b{1, 3};
3 Rational c = a + b;
4 std::cout << a << " " << b << " " << c << "\n";
```

Which of the following statements is the most accurate about the behavior of this code? `B`

A. `operator+` returns a reference to `a`, so after execution, `a` and `c` will have the same value, and `a` will be modified.

B. `operator+` takes its parameters by `const` reference and returns a new `Rational` by value, so neither `a` nor `b` is modified, and `c` receives the sum.

C. `operator+` modifies both `lhs` and `rhs` by reference, so `a`, `b`, and `c` will all have the same value.

D. `operator+` implicitly converts `a` to `double`, adds it to `b` as a `double`, and then implicitly converts the result back to a `Rational`.

> **Solution: B** is correct: `operator+` takes `lhs` and `rhs` by `const` reference and returns a new `Rational` by value. The original `a` and `b` are not modified, and `c` receives the sum `a + b`.

(4) (4') Why is `operator+=` designed as a `member function`?

> **Solution:** `operator+=` modifies the left-hand operand (the object it is called on). As a member function it has direct access to `this` and private data members, and it matches the usual convention that compound assignment operators (like `+=`, `-=`, etc.) are member functions.

(5) (4') In this code, `operator+` is implemented as a non-member friend function. Explain why this design is reasonable, and whether the code supports both expressions `1 + r` and `r + 1` (where `r` is a `Rational`).

> **Solution:** Binary `operator+` is symmetric in its two operands, so making it a non-member allows implicit conversions on both the left-hand and right-hand sides. Here we have a constructor `Rational(int)`, so both `1 + r` and `r + 1` are supported: the `int` is implicitly converted to a temporary `Rational`, and then the non-member `operator+` is called with two `Rational` arguments.

(6) (4') Consider the following code:

```
1 Rational r1{3, 4};
2 auto h = r1 + 2.5;
```

When compiling this code with `g++`, we see an error message similar to the following:

```
./main.cpp: In function 'int main()':
./main.cpp:50:17: error: ambiguous overload for 'operator+' (operand types
are 'Rational' and 'double')
   50 |     auto h = r1 + 2.5;
```

```
      |              ~~ ^ ~~~
      |              |    |
      |              |    |double
      |              Rational
./main.cpp:50:17: note: candidate: 'operator+(double, double)' (built-in)
   50 |     auto h = r1 + 2.5;
      |              ~~~^~~~
./main.cpp:40:10: note: candidate: 'Rational operator+(const Rational&,
const Rational&)'
   40 | Rational operator + (const Rational &lhs, const Rational &rhs)
      |          ^~~~~~~~
```

Explain why this code may result in a compile-time error (overload resolution ambiguity). Suggest one reasonable change to the `Rational` interface to resolve the problem.

Solution: There are two competing ways to form `r1 + 2.5`:

- Convert `2.5` to `Rational` using `Rational(double)` and call `Rational operator+(Rational, Rational)`.
- Convert `r1` to `double` using `operator double()` and then use the built-in `double + double`.

Both require a user-defined conversion, so overload resolution can be ambiguous.
One possible fix is to make `Rational(double)` `explicit` to prevent implicit conversion from `double` to `Rational`, or make `operator double()` `explicit` to prevent implicit conversion from `Rational` to `double` or to remove `operator double()` and instead provide an explicit `to_double()` member function.

6. (40 points) Standard Template Library (STL) and iterators

(1) (5') Which of the following statements regarding STL algorithms is **CORRECT**?

```
struct Box { int size; }; // Note: No operators are overloaded for Box.
std::vector<int> vec = {10, 20, 30, 20};
std::list<int> lst = {10, 20, 30, 20};
std::vector<Box> boxes = {{1}, {2}, {3}};
```

A. `auto pos = std::find(vec.begin(), vec.end(), 20);` makes `pos` point to `vec[3]`.
B. `std::sort(lst.begin(), lst.end());` compiles successfully and sorts the linked list.
C. `std::sort(boxes.begin(), boxes.end());` fails to compile because `Box` does not define the `<` operator.
D. `std::max_element(vec.begin(), vec.end());` returns the value `30`, not an iterator.

Solution: Correct answer: C.
A: `std::find` returns an iterator, points to `vec[1]`.
B: `std::sort` requires a RandomAccessIterator; `std::list` only has bidirectional iterators.
C: `std::sort` needs `operator<` (or a comparator); `Box` does not define it.
D: `std::max_element` returns an iterator, not the value.

(2) (5') Explain what the `capacity` of a `std::vector` means and how it is different from its `size`.

Solution: The `size` of a `std::vector` is the number of elements currently stored in it, while the `capacity` is the amount of memory the vector has already allocated for future elements. The capacity is always at least the size. When more space is needed, the vector increases its capacity (usually by doubling), which may involve allocating a new block of memory and copying the existing elements.

(3) (5') Consider the following program:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec;
    vec.reserve(1000); // (1) Preallocate memory for 1000 elements
    for (int i = 0; i < 1000; ++i) {
        vec.push_back(i);
    }
}
```

Line (1) calls `reserve` to preallocate enough memory so that no reallocations are needed during the loop. If we remove this line, the program still works correctly, but may run less efficiently. Why?

Solution: Calling `reserve(1000)` ensures that the vector has enough memory to store all 1000 elements, so no reallocations occur during the loop.

If we remove the `reserve` call, the vector begins with small capacity and will repeatedly reallocate a larger block of memory as elements are inserted (typically doubling the capacity each time). Each reallocation requires copying all existing elements, which increases the overall running time even though the result is functionally the same.

(4) (5') Consider the following code:

```
std::vector<int> v = {1, 2, 3};
for (auto x : v) {
    x *= 2;
}
```

After this loop, we print `v` and still get `{1, 2, 3}` instead of `{2, 4, 6}`. Briefly **explain** why the loop does not change the contents of `v`, and how to **modify** this so that all elements in `v` are actually doubled.

Solution: The range-based for loop uses `auto x`, so `x` is a copy of each element in `v`. Modifying `x` does not modify the vector.

We need to iterate by reference:

```
for (auto& x : v) {
    x *= 2;
}
```

(5) (5') Rewrite the following range-based for loop using an explicit iterator-based loop:

```
std::vector<int> v = something();
for (auto x : v) {
    do_something_with(x);
}
```

```
std::vector<int> v = something();
{
    ____auto begin = v.begin();____
    ____auto end   = v.end();____
    for (____; begin != end; ++begin____) {
        do_something_with(____*begin____);
    }
}
```

(6) We now implement a singly linked list of `int`. The node type is:

```
struct Node {
    int value;
    Node* next;
};
```

Currently we traverse it like this:

```
void print_list(Node* head) {
    for (Node* p = head; p != nullptr; p = p->next) {
        std::cout << p->value << ' ';
    }
}
```

We would like to be able to write:

```
IntList lst = /* ... */;
for (int x : lst) {
    std::cout << x << ' ';
}
```

To support this, we define an iterator `ListIterator` and a wrapper class `IntList`.

i. (5') Can `ListIterator` for this singly linked list be a **Random Access Iterator**? Briefly explain why or why not.

> **Solution:** No. A singly linked list is not stored contiguously in memory, and the only way to move forward is by following `next` one node at a time. Advancing `n` steps takes O(n) time. Random Access Iterators require constant-time arithmetic like `it + n` and subscripting `it[n]`, which this iterator cannot provide. It is at most a Forward Iterator.

ii. (6') Complete the implementation of `ListIterator` so that it can be used in a range-based for loop:

```
class ListIterator {
    Node* m_ptr;
public:
    explicit ListIterator(Node* p) : m_ptr(p) {}

    // 1. Dereference operator: returns reference to current value
    int& operator*() const {
        return ____m_ptr->value____;
    }

    // 2. Prefix increment: move to next node
    ListIterator& operator++() {
        if (m_ptr) {
            m_ptr = ____m_ptr->next____;
        }
        return *this;
    }

    // 3. Inequality: compare underlying pointers
    bool operator!=(const ListIterator& other) const {
        return _m_ptr != other.m_ptr_;
    }
};
```

iii. (4') Complete the `begin()` and `end()` member functions of `IntList` so that the range-based for loop above is valid:

```
class IntList {
    Node* head;
```

```
public:
    // ...

    ListIterator begin() {
        return _____ListIterator(head)_____;
    }

    ListIterator end() {
        return ___ListIterator(nullptr)___;
    }
};
```